ChatGPT a dit :

Gastronomie Créative (GC) — Bilan des spécifications et des choix techniques

État du projet au 28 octobre 2026 · Synthèse technique et architecturale

Voici un état des lieux complet de ton projet **Gastronomie Créative**, en reprenant les technologies choisies, l'architecture de l'application, la base de données, les fonctionnalités, les aspects de sécurité et les décisions prises au cours du développement.

Je vais distinguer trois situations :

- **Implémenté et vérifié** : nous avons du code ou des tests qui confirment le fonctionnement.
- **Implémenté, mais à valider** : le code existe, mais son fonctionnement complet n'est pas encore confirmé.
- **Prévu ou à préciser** : fonctionnalité ou choix identifié, sans validation suffisante de son implémentation.

Cette distinction sera particulièrement utile pour ton futur dossier de projet CDA.

1. Présentation générale

Gastronomie Créative

Application web de recettes culinaires

Architecture **MVC côté serveur**	Langage principal **C#**
Framework **ASP.NET Core MVC — .NET 10**	Base de données **PostgreSQL**
Interface **Razor, HTML, CSS, JavaScript**	Environnement **Windows 11 / VS Code / Docker**

Objectif fonctionnel

GC est un site consacré à la cuisine, conçu autour de la consultation, de la recherche et de la publication de recettes.

Le concept s'inspire de plateformes culinaires telles que Marmiton et 750g, avec une identité graphique propre, davantage orientée vers une présentation gastronomique élégante.

Les fonctionnalités identifiées dans le périmètre du projet sont :

- Consultation des recettes et de leurs détails.
- Recherche de recettes.
- Organisation des recettes par catégories.
- Création de compte, connexion et espace personnel.
- Création et publication de recettes.
- Gestion des ingrédients, quantités et étapes de préparation.
- Consultation et dépôt d'avis, avec notation.
- Rubriques éditoriales « Actualités » et « C'est la saison ! ».
- Formulaire de contact.

Toutes ces fonctionnalités ne doivent pas être considérées comme terminées : certaines correspondent au périmètre fonctionnel défini et d'autres sont déjà présentes dans le code.

2. Choix du framework : ASP.NET Core MVC

Le choix technique structurant du projet est l'utilisation de **ASP.NET Core MVC avec .NET 10**.

Ce framework permet de développer une application web dynamique en C#, en organisant le code selon le modèle MVC.



Entity Framework Core → PostgreSQL

Accès aux données et persistance

Pourquoi ce choix est pertinent

L'architecture MVC permet notamment :

- De séparer la présentation, les traitements et les données.
- De faciliter la maintenance du projet.
- D'organiser le développement par fonctionnalités.
- D'utiliser des ViewModels pour contrôler les données échangées avec les formulaires.
- D'exploiter Entity Framework Core pour accéder à PostgreSQL.
- De bénéficier des mécanismes intégrés d'ASP.NET Core pour la validation, l'authentification et la protection des formulaires.

Point important : GC n'est pas une application React, Angular ou Vue. L'interface repose principalement sur des vues Razor générées côté serveur, enrichies par du JavaScript natif pour les interactions dynamiques.

C'est un choix cohérent pour une application MVC dans laquelle les pages sont générées par le serveur et les interactions JavaScript restent ciblées.

3. Organisation du projet

Le projet principal s'appelle `GC_MVC`.

L'organisation observée comprend notamment :

- **GC_MVC/**
  - `Controllers/` — actions MVC et traitements HTTP
  - `Models/` — entités métier et de base de données
  - `ViewModels/` — données destinées aux vues et formulaires
  - `Views/` — fichiers Razor `.cshtml`
  - `Data/` — contexte Entity Framework Core
  - `wwwroot/` — CSS, JavaScript, images et ressources statiques
  - `Program.cs` — configuration de l'application
  - `GC_MVC.csproj` — configuration du projet .NET

Le contrôleur `RecettesController.cs` contient notamment des actions pour :

Action	Rôle
`Detail(int id)`	Afficher une recette, ses ingrédients, ses étapes et ses avis
`MesRecettes()`	Récupérer les recettes de l'utilisateur connecté
`CreerRecette()` GET	Charger les données et afficher le formulaire
`CreerRecette(model)` POST	Valider et enregistrer une nouvelle recette

L'organisation par contrôleurs spécialisés est également visible pour les utilisateurs, l'authentification, les ingrédients, les catégories, les avis et d'autres domaines fonctionnels.

4. Choix de la base de données : PostgreSQL

Le système de gestion de base de données choisi est **PostgreSQL**, une base relationnelle.

La base utilisée s'appelle `GC`.

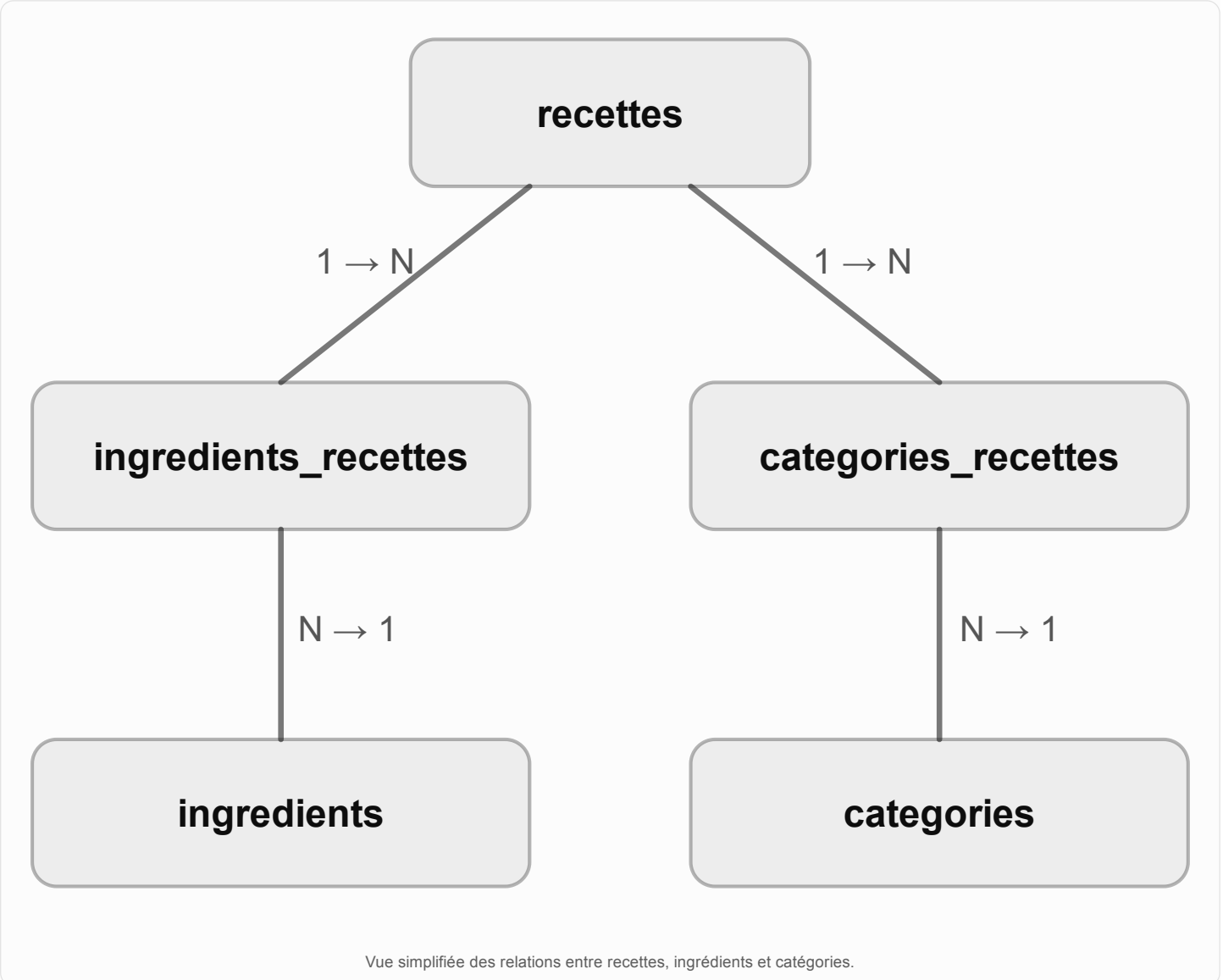
Structure connue

Table	Responsabilité
`utilisateurs`	Comptes et informations des utilisateurs
`recettes`	Informations principales des recettes

`ingredients`	Catalogue des ingrédients
`ingredients_recettes`	Association ingrédients–recettes et quantités
`categories`	Catégories culinaires
`categories_recettes`	Association catégories–recettes
`etapes`	Étapes de préparation
`avis`	Avis et notes des utilisateurs

Le schéma est relationnel : une recette peut être associée à plusieurs ingrédients et à plusieurs catégories.

Relations principales



Vue simplifiée des relations entre recettes, ingrédients et catégories.

Les tables d'association permettent de modéliser des relations plusieurs-à-plusieurs.

La table `ingredients_recettes` contient notamment :

```sql
id_recette
id_ingredient
quantite
```

Le choix d'une quantité textuelle permet de saisir des valeurs comme `300 g`, `2 cuillères` ou `une pincée`. C'est souple pour l'utilisateur, mais moins adapté à des calculs automatiques de quantités ou à des conversions d'unités.

Accès aux données

GC utilise **Entity Framework Core**, avec un contexte nommé `GCContext`.

Exemple :

```C#
List<Ingredients> ingredients = _context.Ingredients.ToList();
```

L'utilisation d'un ORM évite d'écrire manuellement toutes les requêtes SQL et permet de manipuler des entités C#.

Le projet utilise également des requêtes LINQ, notamment des jointures pour construire les détails d'une recette.

Transactions

L'enregistrement d'une recette s'appuie sur une transaction explicite :

```C#
_context.Database.BeginTransaction();
```

L'objectif est de traiter comme un ensemble cohérent la création de la recette et de ses associations.

En cas d'erreur, un `Rollback()` est prévu. En cas de réussite, la transaction est validée par `Commit()`.

C'est un choix pertinent pour protéger la cohérence des données.

5. Choix du frontend : Razor, HTML, CSS et JavaScript

L'interface utilise plusieurs technologies complémentaires.

Technologie	Utilisation
HTML5	Structure des pages et formulaires
CSS3	Mise en page, responsive design et identité visuelle
Razor `.cshtml`	Génération dynamique des pages côté serveur
JavaScript natif	Recherche interactive, manipulation du DOM et formulaires dynamiques
Figma	Conception des maquettes et réflexion sur l'interface

Pourquoi du JavaScript natif ?

Pour les interactions actuellement développées, JavaScript natif suffit sans nécessiter de framework frontend supplémentaire.

Nous avons notamment développé la sélection dynamique des ingrédients :

```JavaScript
const IngredientsFiltres = IngredientsDisponibles.filter(function(ingredient)
{
    return ingredient.nom.toLowerCase().includes(Recherche);
});
```

Puis la création des champs HTML nécessaires à ASP.NET Core MVC :

```JavaScript
IdIngredientHTML.name =
    `IngredientsSelectionnes[${IndexIngredient}].IdIngredient`;

QuantiteIngredientHTML.name =
    `IngredientsSelectionnes[${IndexIngredient}].Quantite`;
```

Ces noms de champs permettent au *Model Binding* de reconstruire automatiquement une liste d'objets C# à partir du formulaire.

Ce fonctionnement a été vérifié avec le débogueur VS Code, qui a reçu trois ingrédients avec leurs identifiants et leurs quantités.

6. Choix des ViewModels

Le projet distingue les entités de la base de données des données nécessaires aux formulaires et aux vues.

C'est un choix architectural important.

Par exemple :

```C#
public class IngredientRecetteViewModel
{
    public int IdIngredient { get; set; }
    public string Quantite { get; set; } = string.Empty;
}
```

Ce ViewModel ne représente pas directement une table. Il sert à transporter les informations saisies par l'utilisateur.

Nous avons également identifié :

- `RecetteCreationViewModel` pour la création d'une recette.
- `RecetteDetailViewModel` pour l'affichage détaillé.
- `IngredientQuantiteViewModel` pour présenter les ingrédients et leurs quantités.
- `AvisUtilisateurViewModel` pour présenter les avis associés à une recette.

Les avantages sont la séparation des responsabilités, la limitation des données exposées aux vues et la possibilité d'appliquer des validations adaptées aux formulaires.

7. Authentification et sécurité

Plusieurs mécanismes de sécurité sont visibles dans le code actuel.

Protection des actions

L'attribut `[Authorize]` protège notamment la création de recettes et la consultation des recettes personnelles.

Protection contre les requêtes CSRF

L'action POST de création utilise `[ValidateAntiForgeryToken]`.

Identification du créateur

L'identifiant de l'utilisateur connecté est récupéré côté serveur à partir de ses claims, plutôt que de faire confiance à un identifiant envoyé librement par le formulaire.

Validation côté serveur

Le contrôleur utilise `ModelState.IsValid` et vérifie l'existence des catégories et des ingrédients avant leur association.

Points de sécurité restant à auditer

Nous n'avons pas encore vérifié intégralement les éléments suivants :

- La méthode de hachage et de vérification des mots de passe.
- La configuration exacte des cookies d'authentification.
- La gestion des rôles et des autorisations avancées.
- La protection des secrets et des chaînes de connexion.
- Les règles de validation de tous les champs.
- La configuration HTTPS et les paramètres de sécurité pour une future mise en production.

Ces éléments ne sont pas nécessairement absents du projet : **ils nécessitent une vérification spécifique**.

8. Environnement de développement et outils

Outil	Choix et utilisation

Windows 11	Environnement de développement
Visual Studio Code	Édition, compilation et débogage
SDK .NET 10	Compilation et exécution de GC
Docker Desktop	Exécution du serveur PostgreSQL
PostgreSQL	Stockage des données
DBeaver	Administration et interrogation SQL
Git	Gestion des versions
GitHub	Hébergement et synchronisation du code
Figma	Maquettes de l'interface

Choix de Docker

PostgreSQL est exécuté dans un conteneur Docker nommé `gc-postgres`.

Ce choix facilite la gestion de l'environnement de base de données, sans devoir installer PostgreSQL directement comme service Windows.

Nous avons également utilisé les outils PostgreSQL pour sauvegarder et restaurer la base de données entre deux ordinateurs :

```bash
pg_dump
pg_restore
```

Avec des commandes Docker pour transférer les sauvegardes vers le conteneur.

Ce qui a été validé : le projet ASP.NET Core MVC et la base PostgreSQL ont pu être remis en fonctionnement sur le deuxième ordinateur.

Gestion des versions

Le projet utilise Git et GitHub pour le suivi du code source.

Nous avons également abordé l'utilisation de `.gitignore` afin d'exclure notamment les répertoires générés par .NET, comme `bin/` et `obj/`, ainsi que les fichiers locaux ou sensibles qui ne doivent pas être versionnés.

9. Choix graphiques et identité visuelle

La direction artistique de GC repose sur une identité culinaire élégante, chaleureuse et contemporaine.

GC

Positionnement graphique

Gastronomie raffinée, minimalisme, lisibilité et absence de surcharge décorative.

Blanc cassé	Beige clair	Or champagne*	Vert naturel*
#FFFDF8	#E5DED1	#C5A879	#4F5945

*Les deux dernières teintes sont illustratives de la direction artistique ; leurs codes hexadécimaux exacts restent à confirmer dans les fichiers CSS et la charte graphique.

Typographie

La police principale retenue est **Montserrat**, avec Inter comme alternative envisagée pour certains usages.

Ressources graphiques

Nous avons travaillé sur :

- Un logo principal Gastronomie Créative.
- Des variantes du logo, notamment en or champagne.
- Des illustrations culinaires.
- Une illustration de couverture.
- Des icônes de réseaux sociaux adaptées à l'identité GC.
- Des éléments décoratifs végétaux.
- Un système de notation visuelle par étoiles.

La cohérence entre le site, les illustrations, les logos et le futur dossier de projet a été une priorité.

10. État des fonctionnalités

Ce tableau distingue ce que nous avons effectivement vérifié de ce qui nécessite encore des tests.

Fonctionnalité	État actuel
Connexion à PostgreSQL	Vérifié
Exécution de l'application ASP.NET Core MVC	Vérifié
Chargement des ingrédients depuis PostgreSQL	Vérifié
Recherche et sélection dynamique des ingrédients	Vérifié
Transmission des quantités au contrôleur	Vérifié
Création d'une recette en BDD	Vérifié
Vérification de l'existence des ingrédients	Code présent
Enregistrement des associations d'ingrédients	À déboguer
Affichage du message de succès	À corriger
Affichage détaillé des recettes	Code présent
Consultation des recettes personnelles	Code présent
Avis et notes	Code présent en partie
Rubriques éditoriales et contact	À vérifier

Les états « Code présent » ne signifient pas qu'une fonctionnalité a été entièrement testée de bout en bout.

11. Points techniques restant à résoudre ou à améliorer

Voici les priorités que je retiendrais pour la suite du projet.

1. **Résoudre l'enregistrement des ingrédients**

   C'est notre priorité immédiate. La recette n° 63 a été créée, mais la requête SQL sur `ingredients_recettes` n'a retourné aucune association.

2. **Améliorer la gestion des erreurs**

   Recharger systématiquement les listes nécessaires au formulaire lorsqu'une validation échoue, et préserver autant que possible les saisies de l'utilisateur.

3. **Afficher les messages de confirmation**

   Vérifier que `TempData["MessageSucces"]` est effectivement affiché dans la vue de destination après l'enregistrement.

4. **Compléter les tests fonctionnels**

   Vérifier les cas nominaux et les erreurs : champs vides, ingrédients supprimés, quantités invalides, doublons et échecs de transaction.

5. **Préparer la documentation technique**

   Formaliser le schéma de données, les règles métier, l'architecture, les dépendances et les procédures d'installation.

12. Synthèse des choix techniques pour ton dossier CDA

Choix	Justification technique
ASP.NET Core MVC	Séparation des responsabilités et organisation du code
C# / .NET 10	Développement du backend et utilisation de l'écosystème .NET
Razor	Production de vues dynamiques côté serveur
JavaScript natif	Interactions dynamiques sans framework frontend supplémentaire
PostgreSQL	Stockage relationnel structuré
Entity Framework Core	Accès aux données par entités C# et LINQ
ViewModels	Séparation des données métier et des données de présentation
Transactions	Cohérence des opérations d'enregistrement
Docker	Gestion reproductible de l'environnement PostgreSQL
Git / GitHub	Versionnement et transfert du code
DBeaver	Administration, inspection et tests SQL
VS Code	Développement, compilation et débogage
Figma	Préparation de l'interface et de l'identité graphique

13. Ce qu'il manque pour obtenir une spécification technique définitive

Nous disposons déjà d'une bonne base documentaire. En revanche, pour constituer un véritable document de spécifications techniques complet, il faudrait encore examiner les éléments suivants :

- Le cahier des charges définitif, pour distinguer les exigences obligatoires des fonctionnalités facultatives.
- Le fichier `GC_MVC.csproj`, pour inventorier précisément les dépendances et leurs versions.
- `Program.cs`, pour documenter les services, les middlewares, l'authentification et la configuration.
- `GCContext.cs` et les classes `Models`, pour vérifier le mapping exact de PostgreSQL, les clés et les contraintes.
- Le schéma SQL complet, pour établir un MCD/MLD fiable.
- Les vues et les fichiers CSS, pour vérifier l'accessibilité et le responsive design.
- Les modalités de déploiement, qui ne sont pas encore définies avec suffisamment de précision.

Conclusion

Gastronomie Créative repose sur une architecture MVC classique et cohérente, associant ASP.NET Core MVC, Entity Framework Core et PostgreSQL. Le frontend est construit avec Razor, HTML, CSS et JavaScript natif, tandis que Docker facilite la gestion de l'environnement de base de données.

Le projet possède déjà plusieurs éléments intéressants pour ta formation CDA : séparation en couches, modélisation relationnelle, authentification, validation côté serveur, manipulation du DOM, transactions et débogage.

Le principal problème technique actuellement identifié concerne l'absence d'associations d'ingrédients lors du dernier enregistrement, malgré la présence du code correspondant.

Pour ton dossier professionnel, je te conseille de conserver ce bilan comme une première version de la documentation d'architecture et des choix techniques. Nous pourrons ensuite le transformer en véritable document de spécifications techniques, avec un schéma d'architecture, un modèle de données, un inventaire des fonctionnalités et une justification formalisée des décisions.

Il sera alors important de documenter les choix effectivement réalisés, sans présenter comme terminées des fonctionnalités qui restent à développer ou à tester.